education

Сортировки: введение, порядковые статистики

Лекция

Константин Жуков

Старший инженер по тестированию,
Вконтакте



education

Цель занятия:

Познакомить слушателей
с различными алгоритмами
сортировки и их сложностью,
а также научить находить
порядковые статистики.

Что входит в сортировку?

- $\{a_1, a_2, ..., a_n\} \rightarrow \{a'_1 \leq a'_2 \leq ... \leq a'_n\}$ или $\{a'_1 \geq a'_2 \geq ... \geq a'_n\}$
- Сортируем только те элементы к которым применима операция сравнения.
- Процесс сортировки состоит из операции сравнения и перестановки.
  Так же мы будем прибегать к операции поиска минимума.
- Операция сравнения всегда тяжелее операции перестановки, поэтому
  в некоторых случаях мы будем сравнивать количество перестановок
  и количество сравнений.
- Зачем вообще сортировать? Представление в пользовательском интерфейсе.
  Упрощает поиск.

Ключ сортировки

Элементы сортируемой последовательности могут иметь любые типы данных.
Обязательное условие — наличие ключа.

Например последовательность:
(Москва, 10000000), (Нью-Йорк, 12000000), (Париж, 9000000),
(Токио, 20000000), (Лондон, 10000000), (Дели, 9000000)

Ключ — число жителей.

Устойчивость сортировки

Алгоритм сортировки устойчивый, если он сохраняет относительный порядок элементов.

Начальная последовательность:
(Москва, 10000000), (Нью-Йорк, 12000000), (Париж, 9000000), (Токио, 20000000),
(Лондон, 10000000), (Дели, 9000000)

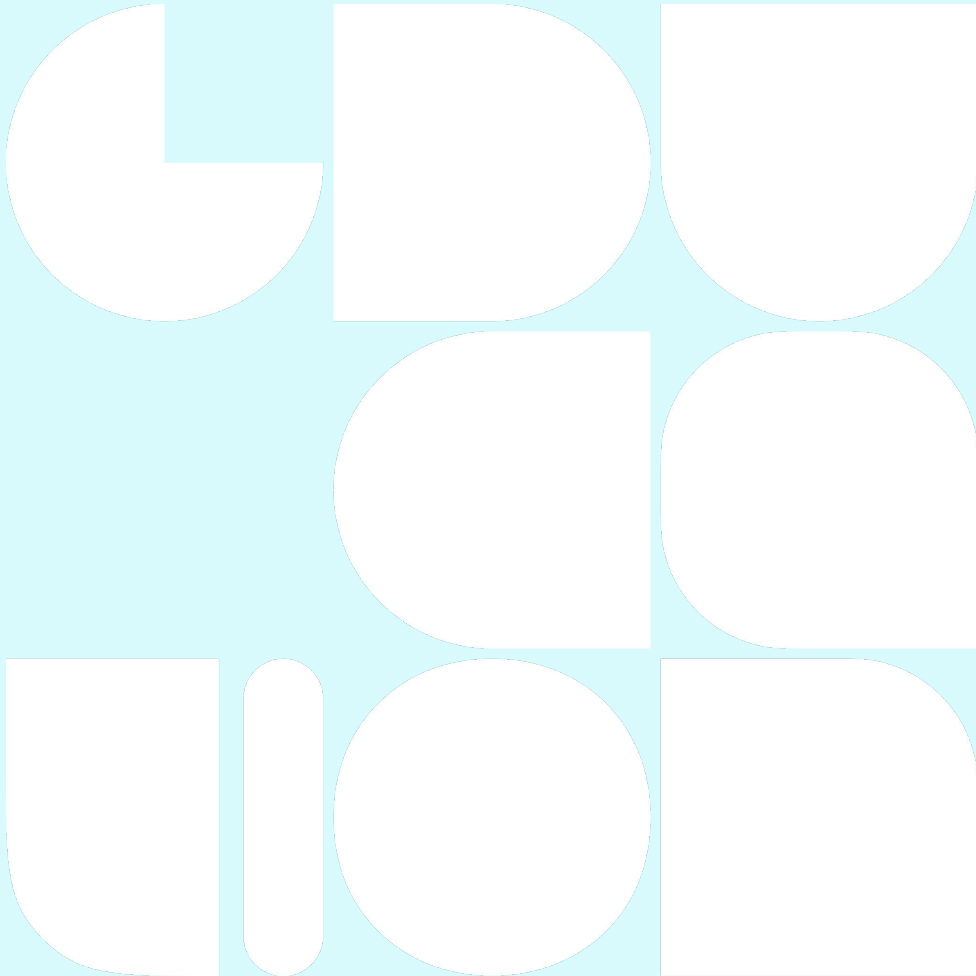
Устойчивая сортировка:
(Токио, 20000000), (Нью-Йорк, 12000000), (Москва, 10000000), (Лондон, 10000000),
(Париж, 9000000), (Дели, 9000000)

Неустойчивая сортировка:
(Токио, 20000000), (Нью-Йорк, 12000000), (Лондон, 10000000), (Москва, 10000000),
(Париж, 9000000), (Дели, 9000000)

Сортировка выбором

Selection sort

Ищем наименьший элемент
в неотсортированной части



Сортировка выбором

Алгоритм

- Делим массив на две части. Слева пустая отсортированная часть, справа не отсортированная.
- На каждой итерации нам необходимо найти минимум в правой части массива и перенести ее в конец левой части.
- После каждого нахождения наименьшего элемента справа сдвигаем границу вправо на 1
- Другими словами на каждой итерации мы увеличиваем отсортированную часть и уменьшаем неотсортированную

Сортировка выбором

Инварианты

{ 5, 3, 15, 7, 6, **2**, 11, 13 }
{ 2, **3**, 15, 7, 6, 5, 11, 13 }
{ 2, 3, 15, 7, 6, **5**, 11, 13 }
{ 2, 3, 5, 7, **6**, 15, 11, 13 }
{ 2, 3, 5, 6, **7**, 15, 11, 13 }
{ 2, 3, 5, 6, 7, 15, **11**, 13 }
{ 2, 3, 5, 6, 7, 11, 15, **13** }
{ 2, 3, 5, 6, 7, 11, 13, **15** }
{ 2, 3, 5, 6, 7, 11, 13, 15 }

Сортировка выбором

- Если в массиве встречается элемент меньше, чем текущий минимальный, то мы обновляем minIndex — индекс минимального элемента.
- После завершения внутреннего цикла, в minIndex хранится индекс минимального элемента в оставшейся части массива. Если этот индекс не равен i (что означает, что минимальный элемент не был уже в правильном месте), мы меняем местами текущий элемент arr[i] с найденным минимальным элементом arr[minIndex].

```
function selectionSort(arr) {
    len = len(arr);

    for (i = 0; i < len - 1; i++) {
        minIndex = i;
        for (j = i + 1; j < len; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            swap(arr[ i], arr[minIndex])
        }
    }

    return arr;
}
```

Сортировка выбором

```
func BenchmarkSelectionSort(b *testing.B) {
    randomArray := generateRandomArray(1000)
    sortedArray := generateSortedArray(1000)
    reversedArray := generateReversedArray(1000)

    b.Run("Random array", func(b *testing.B) {
        for i := 0; i < b.N; i++ {
            b.StopTimer()
            arr := make([]int, len(randomArray))
            copy(arr, randomArray)
            b.StartTimer()
            SelectionSort(arr)
        }
    })

    b.Run("Sorted array", func(b *testing.B) {
        for i := 0; i < b.N; i++ {
            b.StopTimer()
            arr := make([]int, len(sortedArray))
            copy(arr, sortedArray)
            b.StartTimer()
            SelectionSort(arr)
        }
    })
    …
}
```

Сортировка выбором

```
BenchmarkSelectionSort/Random_array-12          4080          295626 ns/op
BenchmarkSelectionSort/Sorted_array-12          4429          269051 ns/op
BenchmarkSelectionSort/Reversed_array-12        4903          243295 ns/op
```

Сортировка выбором

Особенности

- Выбор каждого элемента требует прохода по правой части.
- Для сортировки массива из N элементов требуется N-1 проход
  (Почему -1? Последний элемент, который останется справа будет являться минимальным, так как он единственный)
- На каждой итерации проходим n-i элементов
- Массив может быть уже частично отсортирован и тогда нам потребуются только операции сравнения.

Сортировка выбором

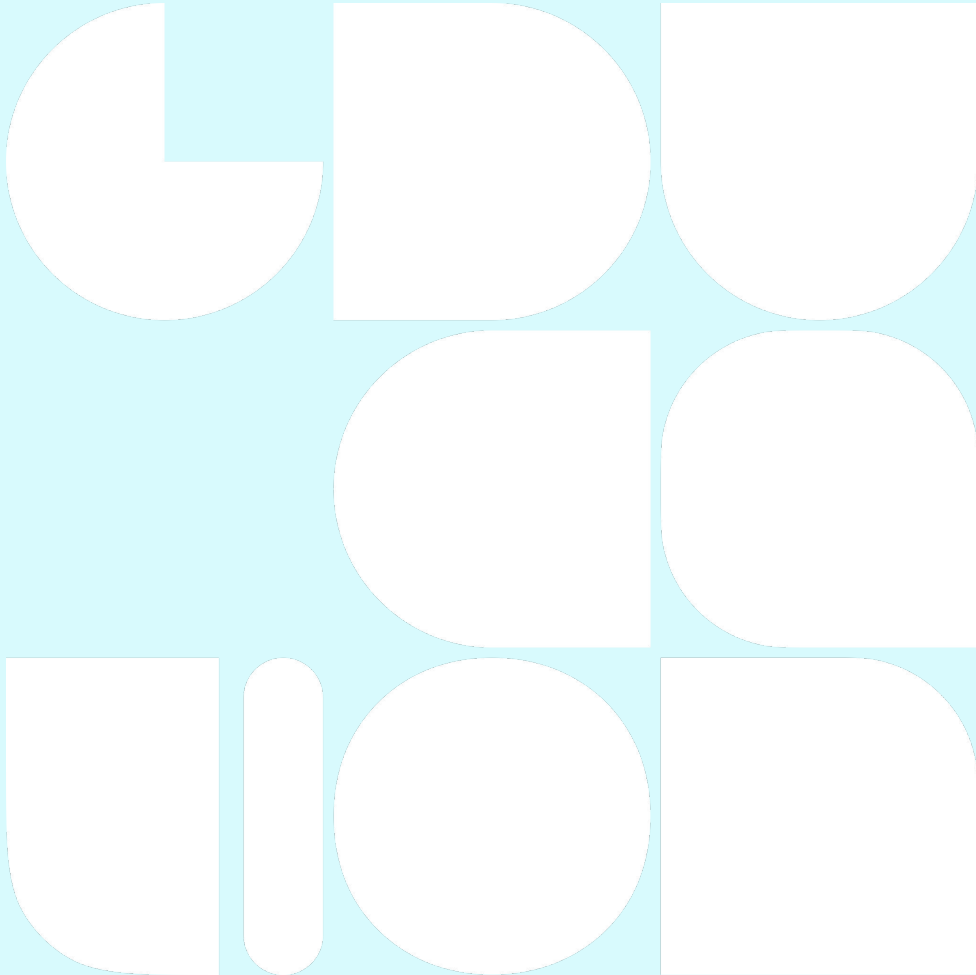
Выводы

- Во всех случаях сложность $O(n^2)$
- Алгоритм устойчив.
- in-place.
- Количество операций обмена O(n) — может пригодиться для сортировок массивов с большими элементами.

Сортировка вставкой

Insertion sort

Берем первый элемент
в неотсортированной части
и вставляем на свое место
в отсортированной части



Сортировка вставкой

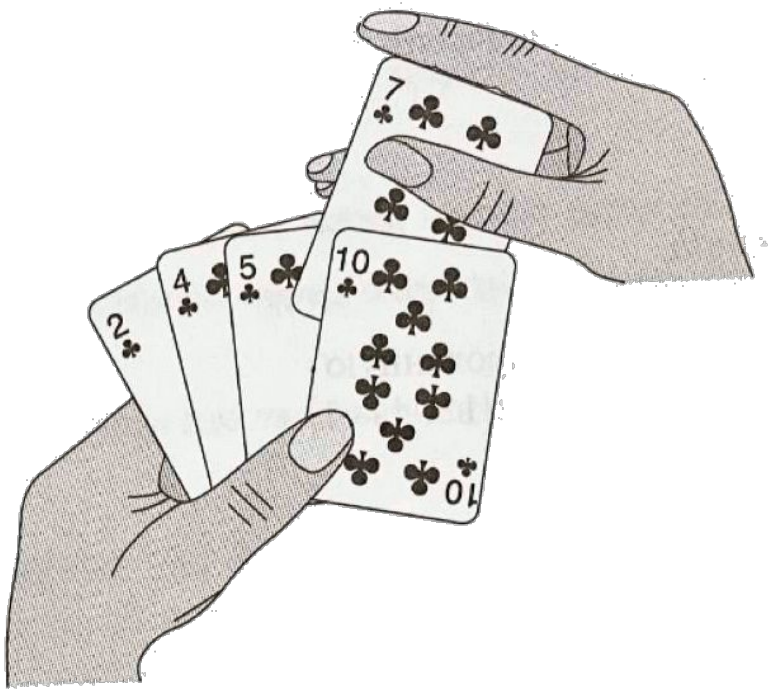
Алгоритм

- Делим массив на две части. Слева отсортированная часть, справа не отсортированная.
- На каждой итерации берем первый не отсортированный элемент, и вставляем в отсортированную часть сдвигая вправо элементы которые больше.
- После вставки сдвигаем границу вправо на 1.
- Другими словами на каждой итерации мы увеличиваем отсортированную часть и уменьшаем неотсортированную.

Сортировка вставкой

Инварианты

{ 5, **3**, 4, 6, 1, 3 }
{ 3, 5, **4**, 6, 1, 3 }
{ 3, 4, 5, **6**, 1, 3 }
{ 3, 4, 5, 6, **1**, 3 }
{ 1, 3, 4, 5, 6, **3** }
{ 1, 3, 3, 4, 5, 6 }



Сортировка вставкой

Наивная реализация.

```
function insertionSort(arr) {
    for i = 1 .. len(arr) {
        j = i
        while j > 0 {
            if arr[j-1] > arr[j] {
                swap(arr[j-1], arr[j])
            }
            j--
        }
    }
}
```

Сортировка вставкой

```
BenchmarkInsertionSort/Random_Array-12          10000          113898 ns/op
BenchmarkInsertionSort/Sorted_Array-12        1212229             995.1 ns/op
BenchmarkInsertionSort/Reversed_Array-12         5706          205717 ns/op
```

Сортировка вставкой

Особенности

- Сортировка упорядоченного массива требует $O(n)$ сравнений.
  Будет произведено 0 перестановок и n-1 сравнение.

- В худшем случае $O(n^2)$, если массив отсортирован по убыванию.
  Количество сравнений тогда равно: $O(n^2)$ <= на 1-ой итерации 1, на 2 - 2, на 3-ей - 3 => 1 + 2 + ... + n-1 = n*(n-1)/2.
  Количество копирований $O(n^2)$ <= 2*(n-1)+(1+2+...+n-1)=2*(n-1)+n*(n-1)/2

- В среднем случае $O(n^2)$

Сортировка вставкой

Выводы

- В среднем сложность $O(n^2)$
- Алгоритм устойчив.
- in-place, число дополнительных переменных не зависит от размера.
- Позволяет упорядочивать массив при динамическом добавлении новых элементов — online-алгоритм.

Сортировка пузырьком

bubble sort

Самый “легкий” элемент всплывает как пузырек воздуха в воде

Сортировка пузырьком

Алгоритм

- Один из простейших в реализации алгоритмов.
- Основная идея: до тех пор, пока соседние элементы не в порядке, меняем их местами.
- Элемент всплывает наверх, обмениваясь с соседними местами. Отсюда и название.

Сортировка пузырьком

Инварианты

{ **5**, **3**, 15, 7, 6, 2, 11, 13}
{ 3, **5**, **15**, 7, 6, 2, 11, 13}
{ 3, 5, **15**, **7**, 6, 2, 11, 13}
{ 3, 5, 7, **15**, **6**, 2, 11, 13}
{ 3, 5, 7, 6, **15**, **2**, 11, 13}
{ 3, 5, 7, 6, 2, **15**, **11**, 13}
{ 3, 5, 7, 6, 2, 11, **15**, **13**}

...

{ 3, 5, 7, 6, 2, 11, 13, 15}
{ 3, 5, 6, 2, 7, 11, 13, 15}
{ 3, 5, 2, 6, 7, 11, 13, 15}
{ 3, 2, 5, 6, 7, 11, 13, 15}
{ 2, 3, 5, 6, 7, 11, 13, 15}

Сортировка пузырьком

```
function bubblesort(arr) {
    len = len(arr);
    for (i = 0; i < len - 1; i++) {
        for (j = 0; j < len - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j+1])
            }
        }
    }
    return arr;
}
```

Сортировка пузырьком

```
function bubblesort(arr) {
    sorted = false
    while !sorted {
        sorted = true
        for i = 0..len(arr)-1 {
            if (arr[i] > arr[i+1]) {
                swap(arr[i], arr[i+1])
                sorted = false
            }
        }
    }
}
```

Сортировка пузырьком

```
BenchmarkBubbleSortBad/Random_Array-12          2558        462135 ns/op          0 B/op          0 allocs/op
BenchmarkBubbleSortBad/Sorted_Array-12          4419        267611 ns/op          0 B/op          0 allocs/op
BenchmarkBubbleSortBad/Reversed_Array-12        3045        408270 ns/op          0 B/op          0 allocs/op
```

```
BenchmarkBubbleSortGood/Random_Array-12          2044        616682 ns/op          0 B/op          0 allocs/op
BenchmarkBubbleSortGood/Sorted_Array-12       1757172           683.8 ns/op          0 B/op          0 allocs/op
BenchmarkBubbleSortGood/Reversed_Array-12        1911        542890 ns/op          0 B/op          0 allocs/op
```

Сортировка пузырьком

Особенности

- Малоэффективный алгоритм, который используется исключительно в тренировочных целях.
- Имеет множество вариаций и улучшений.
- Учитывая, что сравнения долгие операции, а перестановки быстрые, то сортировка вставками более предпочтительна.

Сортировка пузырьком

Выводы

- Крайне проста в реализации и понимании.
- Алгоритм устойчив.
- in-place, сортирует на месте.
- Сложность в наилучшем случае O(n).
- Сложность в наихудшем случае $O(n^2)$

Быстрые сортировки

- Квадратичные устойчивые сортировки слишком медленны для того, чтобы сортировать большие последовательности.
- Мечта: а что, если бы мы имели два отсортированных массива, за какое время можно получить отсортированный массив, содержащий элементы обоих массивов?
- Разделяй и властвуй!

Разделяй и властвуй



Разделяй и властвуй

[]

[8]

[8, 11]

Сортировка слиянием

Merge sort

Слияние — объединение отсортированных массивов.

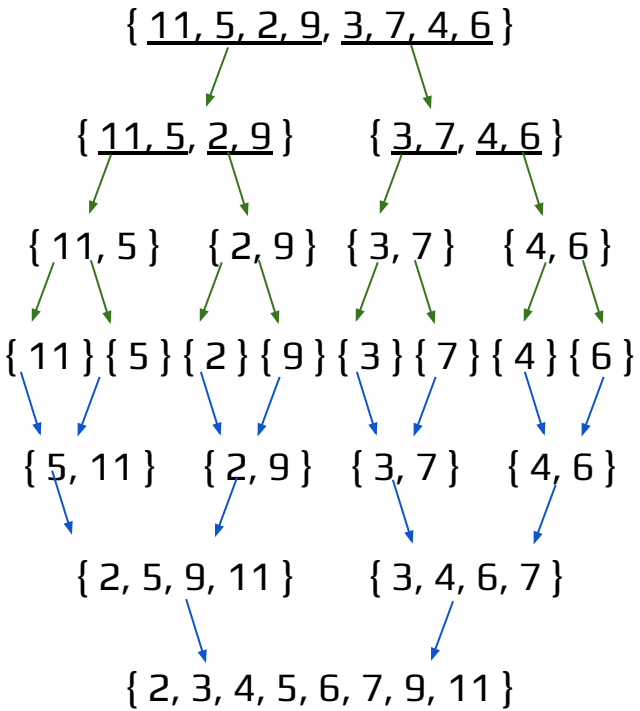
Сортировка слиянием

Алгоритм

- Делим массив пополам.
- Сортируем каждую половину независимо.
- Объединяем две отсортированные последовательности.
- Алгоритм слияния — два указателя.

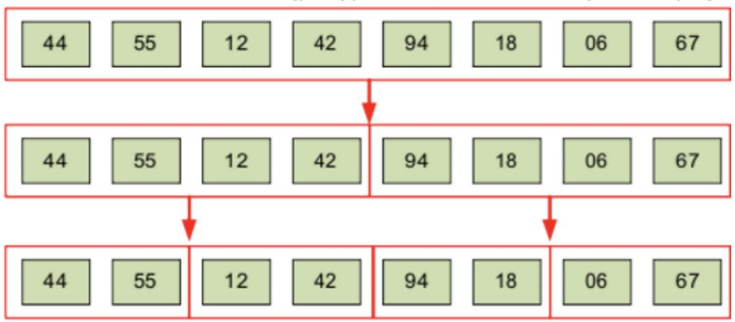
Сортировка слиянием

Инварианты



Сортировка слиянием

Разделяй и властвуй,
нисходящая версия сверху вниз




```
function mergeSort(arr) {
    if len(arr) < 2 {
        return arr
    }
    mid = len(arr) / 2

    leftSide  = arr[0, mid]
    rightSide = arr[mid, len(arr)]

    return merge(mergeSort(leftSide),
                 mergeSort(rightSide))
}
```

Сортировка слиянием

Слияние (два указателя)

```
function merge(a, b) {
    result = [], i = 0; j =  0
    while i < len(a) and j < len(b) {
        if a[i] < b[j] {
            result.append(a[i]);
            i++
        } else {
            result.append(b[j]);
            j++
        }
    }
    while i < len(a) {
        result.append(a[i]); i++
    }
    while j < len(b) {
        result.append(b[j]); j++
    }
    return result
}
```

Сортировка слиянием

```
BenchmarkMergeSort/Random_Array-12          26198          45748 ns/op          81344 B/op          999 allocs/op
BenchmarkMergeSort/Sorted_Array-12          35205          34250 ns/op          81344 B/op          999 allocs/op
BenchmarkMergeSort/Reversed_Array-12        35412          33754 ns/op          81344 B/op          999 allocs/op
```

Сортировка слиянием

Особенности

- На каждой итерации сортировки слиянием массив делится на две половины. Это действие происходит рекурсивно, пока не останутся отдельные элементы. Эта фаза разделения занимает O(log n) операций, так как массив делится пополам на каждом шаге.

- Слияние каждого подмассива занимает O(n) операций, поскольку необходимо сравнить каждый элемент в обоих массивах.

  Сложность = n log n

Сортировка слиянием

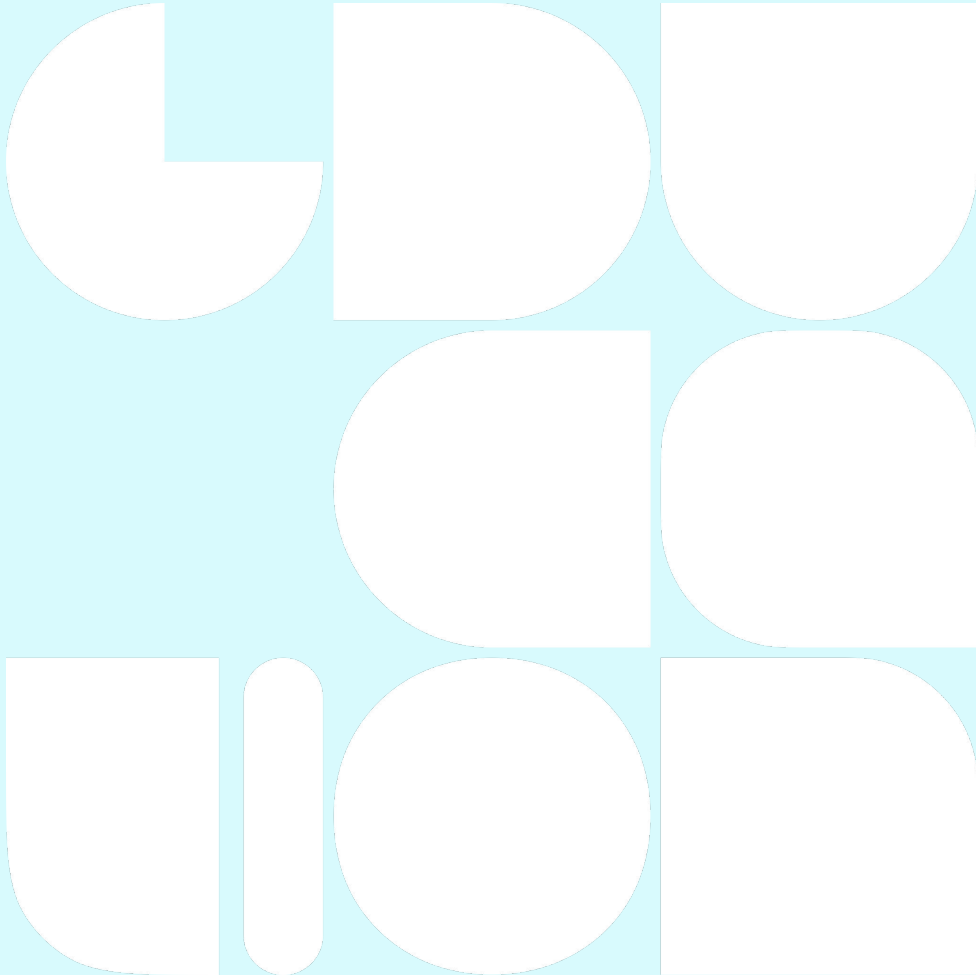
Выводы

- Требует добавочно O(n) памяти.
- Сложность не зависит от входа и равна всегда O(n log n).
- Устойчива!
- Прекрасно подходит для внешней сортировки.
- Прекрасно подходит для параллельной сортировки.

Быстрая сортировка

Quick sort

Выбираем опорный элемент,
все что меньше переносим влево,
все что больше переносим вправо.
Рекурсивно повторяем для каждой части.



Быстрая сортировка

Алгоритм

- Выбираем опорный элемент — pivot.
- Разбиваем массив “in place” на меньшие и больше значения.
- Повторяем алгоритм для большей и меньшей части.

Медиана — элемент, который находился бы в середине упорядоченного массива.
Median({1, 1, 1, 1, 1, 10}) = 1.0
Average({1, 1, 1, 1, 1, 10}) = 2.5

Быстрая сортировка

{7, 3, 9, 1, 5, 8, 2, 6, 4}
0 ... 8

Быстрая сортировка

Инварианты

{7, 3, 9, 1, 5, 8, 2, 6, 4}
 0 4 8

Быстрая сортировка

Инварианты

{**7**, 3, 9, 1, **5**, 8, 2, 6, **4**}
0 4 8

Ищем слева элемент ≥ 5: arr[0]=7.

Ищем справа элемент ≤ 5: arr[8]=4.

Быстрая сортировка

Инварианты

{4, 3, 9, 1, **5**, 8, 2, 6, 7}
0 4 8

Быстрая сортировка

Инварианты

{4, 3, **9**, 1, **5**, 8, **2**, 6, 7}
2 4 6 8

Ищем слева элемент ≥ 5: arr[2]=9.

Ищем справа элемент ≤ 5: arr[6]=2.

Быстрая сортировка

Инварианты

{4, 3, 2, 1, **5**, 8, 9, 6, 7}

Быстрая сортировка

Наивная реализация.
В дальнейшем надо будет реализовать поиск медианы.

- “i” увеличивается, пока не найдет элемент, больший либо равный опорному
- “j” уменьшается, пока не найдет элемент, меньший либо равный опорному
- Когда “i” и “j” определены, если они еще не пересеклись, элементы `**arr[i]**` и `**arr[j]**` обмениваются местами, после чего “i” увеличивается, а “j” уменьшается
- После разделения, быстрая сортировка применяется отдельно к левому и правому подмассивам.
- Если “j” больше 0, сортируется подмассив от начала до “j”.
- Если “i” меньше длины массива, сортируется подмассив от `i` до конца.

```
void qsortRecursive(arr[]) {
    i = 0, j = len(arr)-1
    pivot = arr[len(arr) / 2];
    while(i <= j) {
        while(arr[i] < pivot) i++;
        while(arr[j] > pivot) j--;
        if (i <= j) {
            swap(arr[i], arr[j])
            i++; j--
        }
    }

    if(j > 0)
        qsortRecursive(arr, j + 1);
    if (i < len(arr))
        qsortRecursive(arr[i], len(arr) - i);
}
```

Быстрая сортировка

Особенности

- Худший случай: ведущим выбирается минимальный или максимальный элемент.
- Вероятность такого события крайне мала.
- Один из способов его избежать выбор медианы из трех случайных элементов.

Быстрая сортировка

Выводы

- Может проводиться на месте.
- Сложность в наихудшем случае $O(n^2)$, но с крайне малой вероятностью.
- Сложность в среднем $O(n \log n)$.

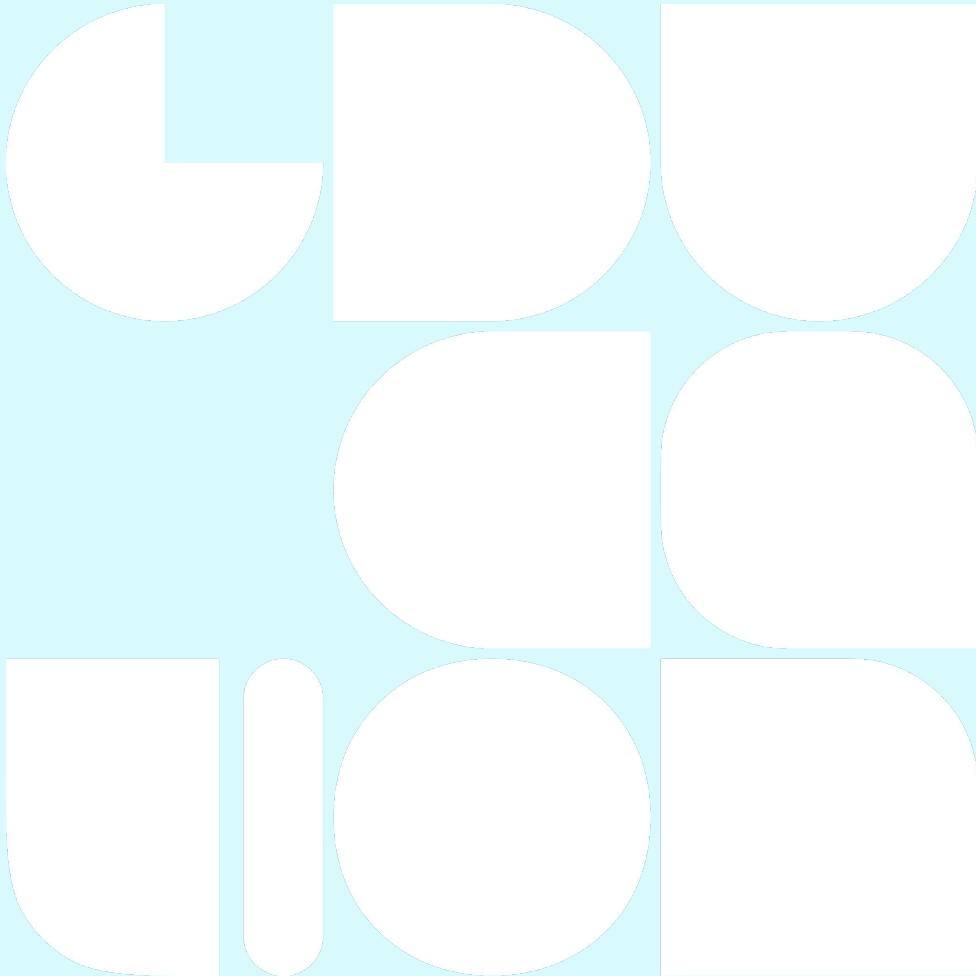
Быстрее быстрой сортировки?

Если использовать особые свойства ключей мы можем сортировать со сложностью меньшей O(nlog n)

Сортировка подсчетом

Counting sort

Подсчитываем количество элементов каждого номинала



Сортировка подсчетом

Алгоритм

- Пусть множество значений ключей ограничено $D(k) = \{k_{min}, \ldots, k_{max}\}$.
- Тогда при наличии добавочной памяти в $|D(k)|$ ячеек сортировку можно произвести за $O(n)$.
- Заранее известно, что значения массива натуральные числа, которые не превосходят 20.
- Заводим массив F[1..20], содержащий счетчики каждого значения.

Сортировка подсчётом

Инварианты

Сортируем массив S = {10, 5, 14, 5, 3, 2, 7, 4, 18, 13, 6, 8}

F_0 = [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]

F_1 = [0 0 0 0 0 0 0 0 0 **1** 0 0 0 0 0 0 0 0 0 0]

F_2 = [0 0 0 0 **1** 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0]

F_3 = [0 0 0 0 1 0 0 0 0 1 0 0 0 **1** 0 0 0 0 0 0]

F_4 = [0 0 0 0 **2** 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0]

...

F_n = [0 1 1 1 2 1 1 1 0 1 0 0 1 1 0 0 0 1 0 0]

S' = {2, 3, 4, 5, 5, 6, 7, 8, 10, 13, 14, 18}

Сортировка подсчётом

- Поиск максимальное и минимальное значение в массиве для настройки диапазона
- Создаём массив countArray, в котором будем хранить количество вхождений каждого элемента
- Проходим по countArray и на основе частоты каждого числа восстанавливаем отсортированный массив.

```go
func countingSort(arr []int) {
    if len(arr) == 0 {
        return
    }
    maxVal, minVal := arr[0], arr[0]
    for _, v := range arr {
        if v > maxVal {
            maxVal = v
        }
        if v < minVal {
            minVal = v
        }
    }
    // Массив для подсчета частоты появления каждого числа
    // От minVal до maxVal включительно
    countArray := make([]int, maxVal-minVal+1)
    // Подсчитываем количество каждого элемента
    for _, v := range arr {
        countArray[v-minVal]++
    }

    // Собираем отсортированный массив
    index := 0
    for i, count := range countArray {
        for count > 0 {
            arr[index] = i + minVal
            index++
            count--
        }
    }
}
```

Сортировка подсчетом

```
BenchmarkCountingSort/Random_Array-12          282108          3897 ns/op          8192 B/op          1 allocs/op
BenchmarkCountingSort/Sorted_Array-12          391230          3038 ns/op          8192 B/op          1 allocs/op
BenchmarkCountingSort/Reversed_Array-12        402103          3099 ns/op          8192 B/op          1 allocs/op
```

Сортировка подсчетом

Выводы

- Ключи должны быть перечислимы.
- Пространство значений ключей должно быть ограниченным.
- Требуется дополнительная память O(|D(k)|).
- Сложность O(n) + O(|D(k)|).

Поразрядная сортировка

Radix sort

Сравнения происходят по каждому разряду отдельно

Поразрядная сортировка

Алгоритм

- Разобьем ключ на фрагменты — разряды и представим его как массив фрагментов.
- Все ключи должны иметь одинаковое количество фрагментов.
- Устойчиво сортируем массив по каждому разряду отдельно.

Поразрядная сортировка

Инварианты

Сортируем массив S = [170, 45, 75, 90, 802, 24, 2, 66]

Сортируем по младшему разряду:
- [170, 90] (0), [802, 2] (2), [24] (4), [45, 75] (5), [66] (6).
- [170, 90, 802, 2, 24, 45, 75, 66]

Сортируем десятки:
- [802, 2] (0), [24] (2), [45] (4), [66] (6), [170, 75] (7), [90] (9).
- Полученный порядок: [802, 2, 24, 45, 66, 170, 75, 90]

Поразрядная сортировка

Сортируем сотни:

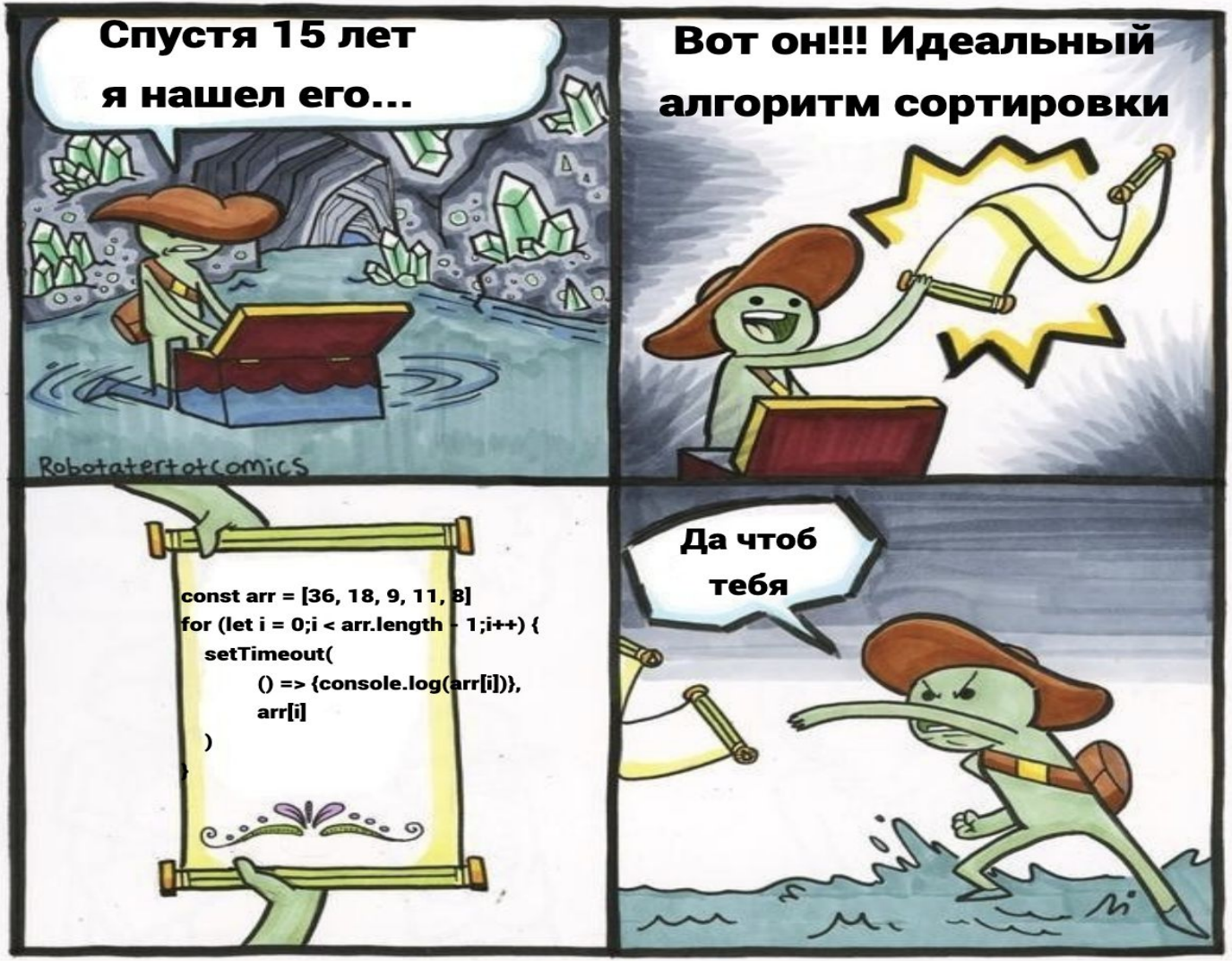
- [2, 24, 45, 66, 75, 90] (0), [170] (1), [802] (8).
- [2, 24, 45, 66, 75, 90, 170, 802]

$S' = [2, 24, 45, 66, 75, 90, 170, 802]$

Поразрядная сортировка

Выводы

- Требует ключи, которые можно трактовать как множество перечислимых фрагментов.
- Требуется дополнительная память $O(D(k_i))$ на сортировку фрагментов.
- Сложность $O(n \cdot D(k_i))$.
- Не универсальна, ее применение ограничено случаями, когда данные хорошо вписываются в модель разрядов
- Может быть крайне эффективной при сортировке больших наборов данных с одинаковой длиной разрядов.


Спустя 15 лет
я нашел его...
Robotatertotcomics
Вот он!!! Идеальный
алгоритм сортировки
const arr = [36, 18, 9, 11, 8]
for (let i = 0;i < arr.length - 1;i++) {
setTimeout(
() => {console.log(arr[i])},
arr[i]
)
}
Да чтоб
тебя

Итоги

Алгоритм	Лучший случай	Средний случай	Худший случай	Доп. память	Устойчивая?
Пузырьком	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$	Да
Шелла	$O(N^{7/6})$	$O(N^{7/6})$	$O(N^{4/3})$	$O(1)$	Нет
Вставкой	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$	Да
Выбором	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$	Нет
Быстрая	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(1)$	Нет
Слияния	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$	Да
Пирамида	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(1)$	Нет
Подсчетом	$O(N)$	$O(N)$	$O(N)$	$O(D)$	Да
Поразрядная	$O(N)$	$O(N)$	$O(N)$	$O(R+N)$	Да

Спасибо за внимание!

education